Audio Deepfake Detection

MODULE 1

01

0:00 – 0:15 · 15 min

Welcome & Context

Icebreaker poll + overview of the two modules.

Section 1 of 7

Two Modules · One Goal: Tell Real from Fake

A six-hour workshop in two modules — generation & attacks first, then detection & evaluation.

MODULE 1 · ~3 HOURS

How Fake Voices Are Made

- Evolution & societal impact of voice synthesis
- Text-to-Speech (TTS): text → waveform
- Voice Conversion (VC): swap the speaker
- Logical Access (LA) & Replay attacks
- The artifacts each method leaves behind

MODULE 2 · ~3 HOURS

How We Detect Them

- The passive detection pipeline
- Features: CQCC / LFCC → Wav2Vec2 / XLS-R
- ASVspoof datasets & train / dev / eval splits
- Evaluation: EER, t-DCF, AUC + generalization
- SincNet → RawNet → AASIST (end-to-end)

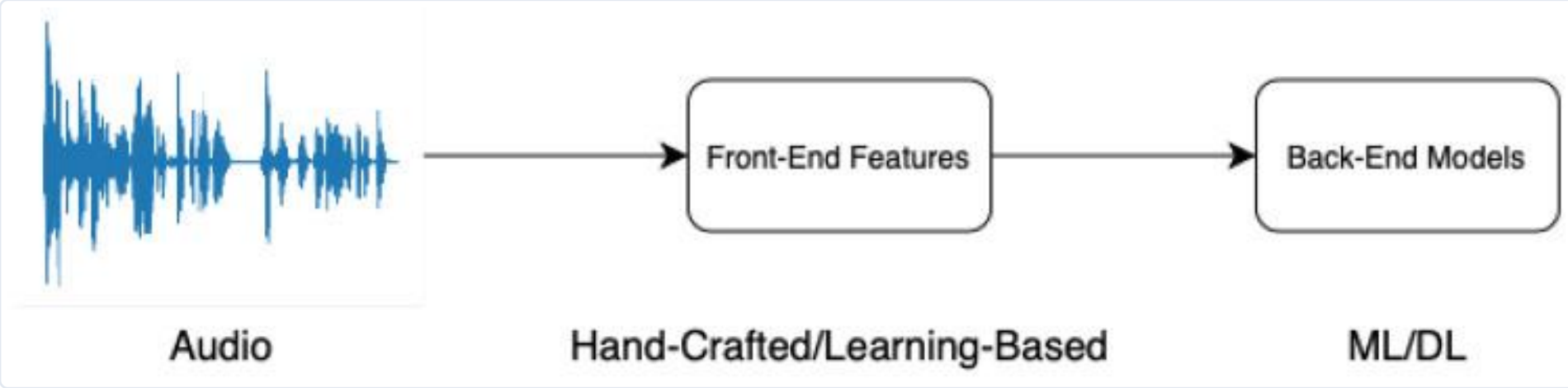
Both modules end with a hands-on lab and a worked example with answers.

By the End, You Will Be Able To…

- Explain how TTS and Voice Conversion synthesise or clone speech — and the spectral, temporal, phase and prosodic artifacts each leaves behind.
- Distinguish the four attack scenarios (Logical Access, Physical Access, Replay, Deepfake) and what signal each exposes.
- Build a passive detector: extract features, train a classifier, and output a real / fake score.
- Evaluate a detector with EER, t-DCF and AUC — and reason about the cross-dataset generalization gap.
- Trace the progression from hand-crafted features → SincNet → RawNet → AASIST toward end-to-end detection.
- Connect detection to real-world security: voice-cloning fraud, biometric spoofing, and robustness under noise & compression.

What is Audio Deepfake Detection?

Audio deepfake detection = automatically deciding whether a voice clip is genuine human speech or AI-synthesised / manipulated audio.

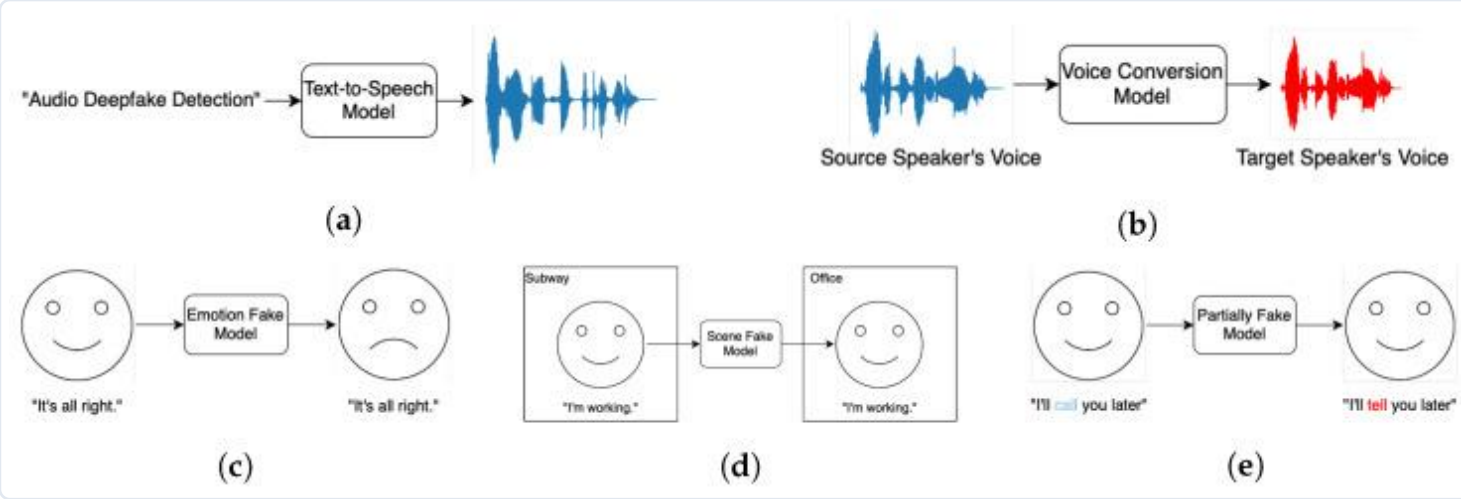


A detector takes one audio clip → extracts features → classifies it as REAL or FAKE. (Zhang et al., Sensors 2025, CC BY 4.0)

- Why automatic? humans correctly spot fake speech only ~73% of the time — too unreliable to guard a phone line or a bank.
- One clip, one decision: the detector outputs a single real/fake verdict in real time — the job of every countermeasure in this course.

What is Audio Deepfake Detection?

To detect a fake, you first have to know how it is made. Two engines create almost all fake speech:



Audio deepfake types: (a) text-to-speech (b) voice conversion (c–e) emotion / scene / partial fakes. (Zhang et al., Sensors 2025, CC BY 4.0)

Text-to-Speech (TTS)

Reads text in a voice that needs no original recording — the whole signal is synthesised from scratch.

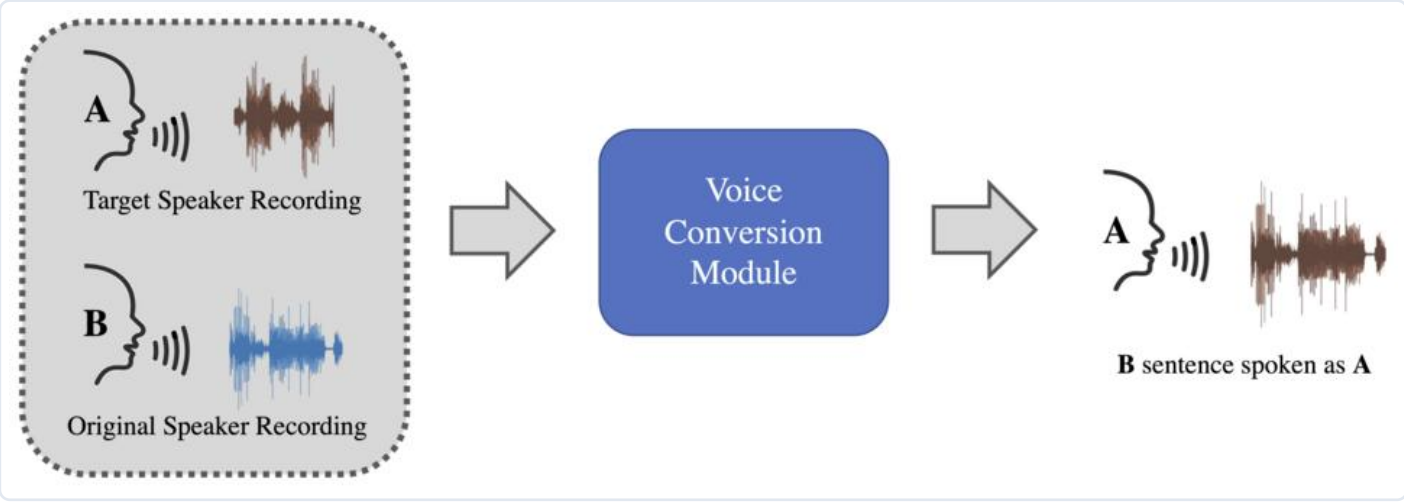
Voice Conversion (VC)

Takes a real recording and re-renders it in someone else's voice — keeps the words, swaps the speaker.

Beyond TTS & VC: emotion / scene / partially-faked audio change only PART of a genuine clip — even harder to catch.

What is Audio Deepfake Detection?

Cloning a voice now needs only seconds of audio — so detection is a security problem, not a curiosity.



Imitation-based audio deepfake generation: a few seconds of a target voice is enough to synthesise entirely new speech. (Wikimedia Commons, CC BY-SA 4.0)

73%

humans correctly spot fake speech — too unreliable to protect anything.

$25.6M

lost in one Hong Kong deepfake 'CFO' video call (2024).

~3 sec

of audio cloned from social media — then it can say anything.

So the task: build a detector that works in real time, on one clip, and generalises to voices it has never seen.

Source: Mai et al. (2023) PLoS ONE 18:e0285333; Hong Kong Police briefing, Feb 2024; diagram: Wikimedia Commons (CC BY-SA 4.0)

MODULE 1

02

0:15 – 0:50 · 35 min

Evolution & Societal Impacts

History of voice synthesis; real-world harms; ethics & regulation.

Section 2 of 7

Laws Restricting Audio Deepfakes — by Jurisdiction

Four jurisdictions, one shared idea: label synthetic audio, get consent, keep provenance.

European Union · AI Act (Reg. 2024/1689) · in force Aug 2024

Art. 50 transparency — deepfake audio/video must be labelled as AI-generated, and altered election media disclosed. Non-compliance fines reach up to €35M or 7% of global turnover.

United States · Federal (FCC / FTC) + State laws · 2024–25

FCC (2024): AI-voice robocalls illegal under the TCPA (fake Biden case). FTC §5 bans deceptive AI voices; Take It Down Act (2025) mandates 48h removal of intimate deepfakes. States (CA, TX, MN) add liability.

China · Deep Synthesis Provisions (CAC) · 2023; marking 2024

AI audio/video must be labelled and watermarked; deep-synthesis providers must verify users' real identity. Using a person's voice or likeness without consent, or publishing misleading synthetic content, is prohibited.

South Korea · PIPA + AI Basic Act · 2020; 2025

Non-consensual deepfakes are a criminal offence (imprisonment + fines); 2024 rules require AI election content to be labelled. The AI Basic Act (2025, effective 2026) adds oversight of high-risk AI and transparency duties.

Source: Reg. (EU) 2024/1689 Art. 50; FCC TCPA declaratory rul. (2024); Take It Down Act Pub.L. 119-12 (2025); CAC Deep Synthesis Provisions (2023); Korea PIPA / AI Basic Act (2025)

MODULE 1

03

0:50 – 1:20 · 35 min

Key Generation Techniques

TTS, Voice Conversion, LA & Replay attacks — and the artifacts each leaves behind.

Section 3 of 7

Three Attack Types We Will Cover

Three ways a voice can be faked — each leaves different clues, so each needs a different detection strategy.

Text-to-Speech

TTS

Synthesises a brand-new voice from written text. No recording of the target is needed — the entire signal is artificial.

Voice Conversion

VC

Takes a real recording and re-renders it in someone else's voice — keeping the words, swapping the speaker.

Replay Attack

REPLAY

The simplest: a genuine human voice is recorded and simply played back to the system. No AI is used, yet it still fools the verifier.

For each: how it is generated → what artefacts it leaves → how a detector catches it.

Text-to-Speech — Three Stages

A modern neural TTS is a three-stage pipeline — each stage shapes how the result sounds.

① Text front-end

Tokens → phonemes (G2P); predict stress & prosody.

② Acoustic model

Phonemes → mel-spectrogram (the voice's time-frequency recipe).

③ Vocoder

Spectrogram → playable waveform (phase + samples).

Source: van den Oord et al. (2016) WaveNet arXiv:1609.03499; Shen et al. (2018) Tacotron 2 arXiv:1712.05884

Tokens & Phonemes — Atoms of Text and Sound

Before a model can speak a word, it first turns the text into units of sound.

EXAMPLE · how the word “speak” becomes sounds

Token — a unit of TEXT

- The chunk of text the model reads at once.
- Whole word: speak → [speak]
- Sub-word (BPE): speak → [sp][eak]
- Characters: speak → s p e a k
- Sub-words cover any word — even unseen ones.

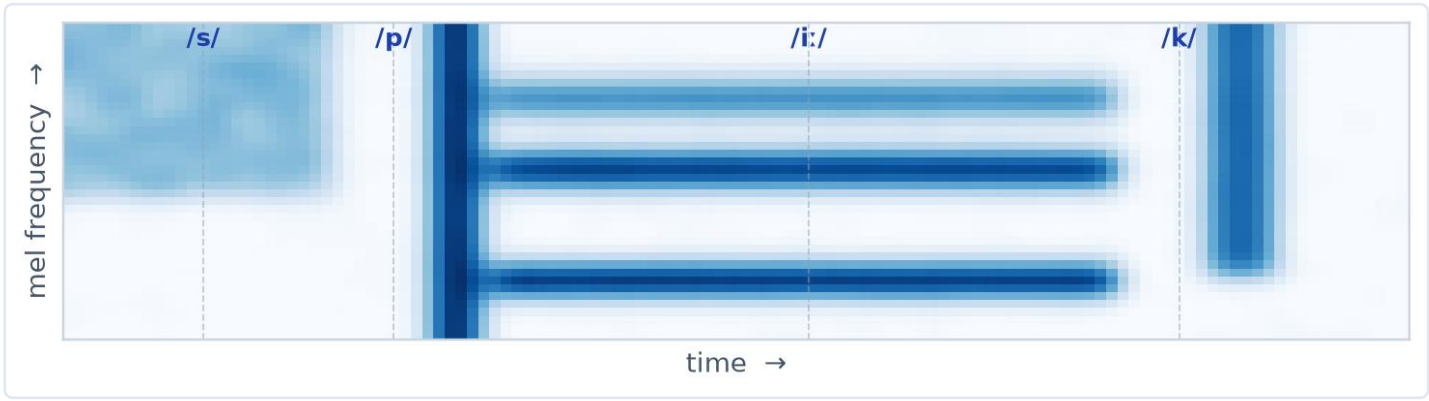
Phoneme — a unit of SOUND

- One speech sound — the atom of pronunciation.
- speak = /s p iː k/
- Letter “c”: /k/ in cat, /s/ in city
- G2P: grapheme → phoneme (spelling → sound)
- Tells the model how to SAY the word.

Source: IPA chart (Intl. Phonetic Association); Sennrich et al. (2016) BPE arXiv:1508.07909; Yao & Zweig (2015) G2P

The Acoustic Model — From Phonemes to a Voice

The acoustic model is the heart of TTS: it maps the phoneme sequence to a mel-spectrogram — a time × frequency picture of the voice.



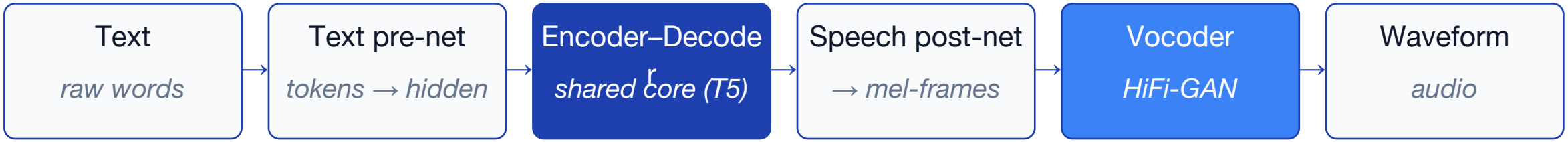
Mel-spectrogram for "speak" — the acoustic model's output. Bright horizontal bands = vowel formants; dashed lines mark the four phonemes /s p iː k/.

What it does: a neural network (Tacotron 2, FastSpeech 2, SpeechT5) that converts the phoneme sequence into a mel-spectrogram — fixing pitch, timing, loudness and timbre. A separate vocoder (HiFi-GAN, WaveNet) then turns that picture into audible waveform samples.

Source: Shen et al. (2018) Tacotron 2 arXiv:1712.05884; Ren et al. (2020) FastSpeech 2 arXiv:2006.04558; Ao et al. (2022) SpeechT5 arXiv:2110.07205

SpeechT5 — A Unified Speech Transformer

TTS is the task — turning text into speech. SpeechT5 is one model that performs it: built on T5's encoder–decoder, it also does ASR, voice conversion and speech enhancement by swapping modular pre-/post-nets around a shared core.



SpeechT5 & TTS

- TTS = the task: text → speech.
- SpeechT5 = one model that does TTS.
- Sits beside Tacotron 2, FastSpeech 2.

Why it is unified

- One encoder–decoder for every task.
- Swap pre-/post-nets to switch task.
- Trained jointly on speech–text pairs.

Deepfake angle

- Open weights: microsoft/speecht5_tts.
- Makes clean, convincing cloned voices.
- Next page: run it in 7 lines.

Source: Ao et al. (2022) SpeechT5: Unified-Modal Encoder-Decoder Pre-Training for Spoken Language Processing, arXiv:2110.07205

Example: Synthesize Speech with SpeechT5

What the code does

- Load: SpeechT5 TTS model + HiFi-GAN vocoder.
- Speaker: an x-vector embedding picks the timbre.
- Generate: text → acoustic model → vocoder → waveform.
- Classic demo: Google Duplex (2018) booked a restaurant by phone; the human could not tell.

tts_demo.py

```python
# Text-to-Speech with SpeechT5
from transformers import (SpeechT5Processor,
    SpeechT5ForTextToSpeech, SpeechT5HifiGan)
from datasets import load_dataset

name = "microsoft/speecht5_tts"
processor = SpeechT5Processor.from_pretrained(name)
model     = SpeechT5ForTextToSpeech.from_pretrained(name)
vocoder   = SpeechT5HifiGan.from_pretrained("...hifigan")

inputs = processor("Hello, class begins now.", return_tensors="pt")
spk = load_dataset("Matthijs/cmu_arctic",
    split="validation")[0]["speaker_embeddings"]
wav = model.generate_speech(inputs["input_ids"], spk, vocoder=vocoder)
```

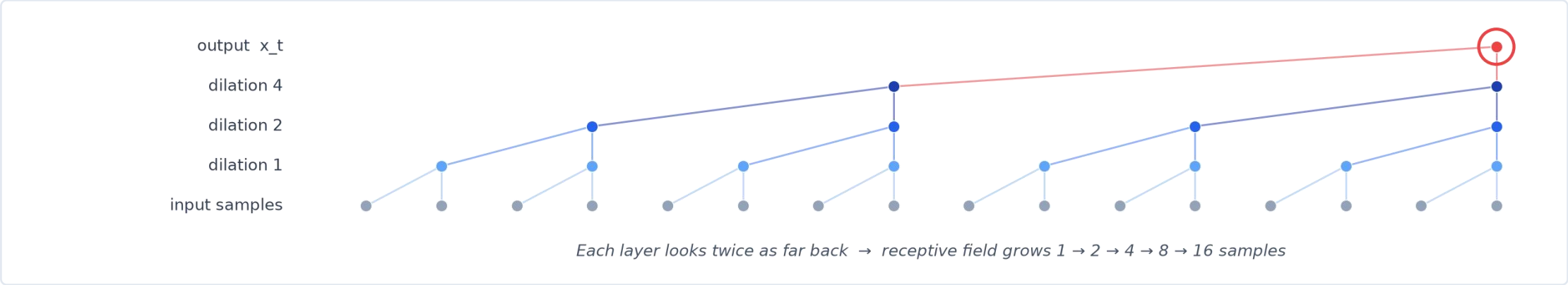
Listen: original vs. algorithm output

Reference voice (real)

SpeechT5 output

WaveNet — Generating Audio One Sample at a Time

WaveNet (DeepMind, 2016) generates raw audio directly — predicting one waveform sample at a time from all past samples, using stacked dilated causal convolutions. It was the first neural net to sound truly human and still powers the vocoder inside TTS pipelines.



Stacked dilated causal convolutions — each layer looks twice as far into the past, so a deep stack covers thousands of samples with few parameters.

Causal + dilated

- Causal: sees only the past → can generate.
- Dilated: each layer doubles the look-back.
- Deep stack → long history, few params.

Autoregressive

- Predicts x_t from all past samples x_<t.
- Sample-by-sample, fed back in.
- Formula on the next slide.

Why it matters

- First neural TTS to sound human (2016).
- Vocoder in Tacotron 2 (mel → wave).
- Backbone of neural voice cloning.

Source: van den Oord et al. (2016) WaveNet: A Generative Model for Raw Audio, arXiv:1609.03499

WaveNet — Autoregressive Generation

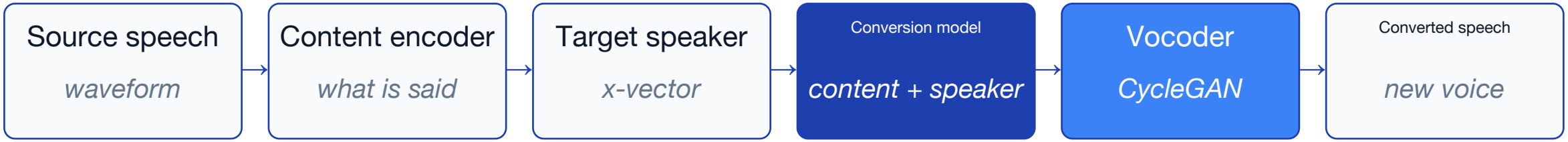
Belongs to: TTS — Text-to-Speech — the generating rule that defines how a neural TTS makes each audio sample.

$$p(x \mid c) = \prod_{t=1}^{T} p(x_t \mid x_{<t},\ c)$$

Symbols	x = waveform; c = condition (text); t = sample index; x_{<t} = past samples; T = total samples; ∏ = product over all t.
How to read it	Generate one sample at a time — each x_t depends only on the past x_{<t} and condition c; multiply all step probabilities for the whole waveform.

Voice Conversion — Change the Speaker, Keep the Words

Voice Conversion re-renders a recording in a different speaker's voice: a content encoder keeps what is said, a target speaker embedding swaps whose voice it is, and a vocoder synthesises a new waveform.



Keep content

- Content encoder keeps the words.
- Features: bottleneck / PPG / mel.
- Speaker-independent — only what is said.

Swap speaker

- Target x-vector sets the new voice.
- Model fuses content + speaker.
- Same words, different identity.

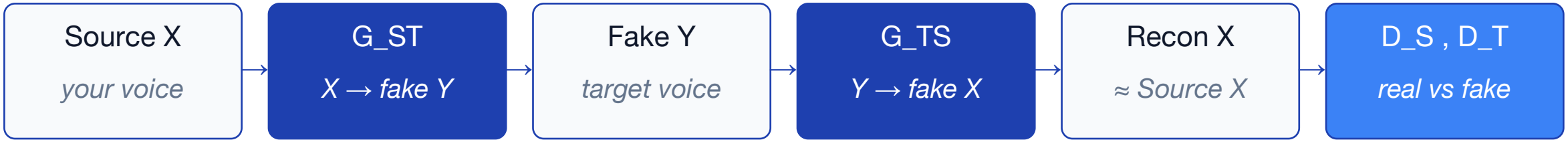
Re-synthesise

- Vocoder renders the fused features.
- Clean waveform in target voice.

Source: Qian et al. (2019) AutoVC arXiv:1905.05879; Li et al. (2021) FreeVC arXiv:2206.08847; CycleGAN-VC / StarGAN-VC → next slides

CycleGAN-VC — Convert Voices Without Parallel Data

CycleGAN-VC converts a source speaker into a target speaker with two generators (source↔target) and two discriminators, tied by cycle-consistency — it needs no parallel recordings (the same sentences spoken by both).



Cycle consistency

- Round-trip G_TS(G_ST(x)) ≈ x.
- Preserves the linguistic content.
- Forces a realistic, faithful conversion.

Adversarial

- D_T pushes Fake Y toward a real target.
- D_S judges reconstructions of X.
- Each D tells real from converted.

No parallel data

- Trains on separate source & target sets.
- Same sentences by both NOT needed.
- Only a few samples of each speaker.

Source: Kaneko & Kameoka (2018) CycleGAN-VC arXiv:1711.11294; Zhu et al. (2017) CycleGAN arXiv:1703.10593

Question 1

A text-to-speech system can read the SAME sentence in many different people's voices.

Question: what does the system change to switch from one speaker to another?

And in Voice Conversion (turning a recording into someone else's voice) — what does it swap?

Answer 1

TTS: it swaps the speaker embedding — a small vector (an “x-vector”) that stands for “whose voice”. Same text + different embedding → different voice.

Voice Conversion: keep the content (z_c = what is said), swap in a new embedding (e_tgt = whose voice), then regenerate the audio.

WaveNet — Joint Probability of a Sequence

Problem. A WaveNet generates 5 audio samples with conditional probabilities 0.9, 0.8, 0.75, 0.6, 0.5 (each depends on the past). (a) What is the joint probability of the whole sequence? (b) Its natural-log probability? (c) A spoof clip scores a joint probability of 0.10 — which clip is more likely to be real?

Worked solution

Step 1. $P(x_1..x_5) = 0.9 \times 0.8 \times 0.75 \times 0.6 \times 0.5 = 0.162$

Step 2. log-prob = $\ln(0.162) = -1.82$

Step 3. Compare the two clips by joint probability (higher = more likely real).

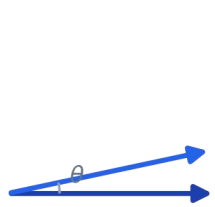
Step 4. 0.162 (clip 1) > 0.10 (spoof) → clip 1 is more likely real.

Answer. Joint P = 0.162 (log-prob −1.82); clip 1 is more likely real than the spoof.

Worked challenge — multi-step quantitative. WaveNet: P(x)=∏ P(x |x_{<t},c) — slide p18.

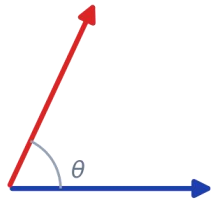
Cosine Similarity — Do Two Voices Point the Same Way?

A neural net turns each voice into a list of numbers — think of it as an arrow. Cosine similarity simply asks: do two arrows point the same way? Same direction = same speaker.



small angle → arrows nearly overlap

cos θ ≈ 0.99 → same voice



large angle → arrows point apart

cos θ ≈ 0.42 → different voice

The score is the cosine of the angle θ between the two arrows.

Same voice

- Two arrows nearly overlap.
- Score close to 1.
- → same speaker.

Different voice

- Arrows point apart.
- Score well below 1.
- → different speaker.

Why direction?

- Ignores loudness and length.
- Only the voice's identity matters.
- So it is robust to volume.

Source: Speaker embeddings (x-vector) + cosine similarity — standard in ASV / ASVspoof

Cosine Similarity — The Score & Catching a Fake

The score is the cosine of the angle between the two arrows — always between –1 and +1. A speaker-verifier compares a new clip to a stored reference: a high score means “same voice”, so a convincing fake scores dangerously high.

$$\cos\theta = \frac{\mathbf{A}\cdot\mathbf{B}}{\|\mathbf{A}\|\,\|\mathbf{B}\|}$$

Top = how much the arrows agree. Bottom = cancels out loudness and length, leaving only direction.

A real repeat of the same speaker sits near +1; a different voice drops toward 0.

How detectors use it: voice-conversion and TTS training try to push this score up (fake → target). A detector flags a clip whose embedding is too close to a known target, yet lacks a real recording’s natural variability.

Source: cosine scoring on speaker embeddings — ASVspoof challenge; used in the verification code on later slides

Question 4

After Voice Conversion, we score how close the result is to the target voice:

1 = identical voice, 0 = completely unrelated.

The score comes out as 0.98.

What does 0.98 tell you? And what would a score of about 0.5 mean?

Answer 4

0.98 (≈ 1): the converted voice is almost identical to the target — the conversion succeeded.

≈ 0.5: only halfway similar — the target's voice did not really transfer; the conversion mostly failed.

Cosine Similarity — Compute It by Hand

Problem. An enrolled speaker has embedding e = (3, 4), so |e| = 5. Three test clips arrive: a = (2, 1), b = (4, 5), c = (1, 3). Compute the cosine similarity of each to e. A clip is accepted as the same speaker only if cosine ≥ 0.95. Which clip(s) pass?

Worked solution

Step 1. cos(e,a) = (3·2 + 4·1) / (5·√5) = 10 / 11.18 = 0.894

Step 2. cos(e,b) = (3·4 + 4·5) / (5·√41) = 32 / 32.02 = 0.999

Step 3. cos(e,c) = (3·1 + 4·3) / (5·√10) = 15 / 15.81 = 0.949

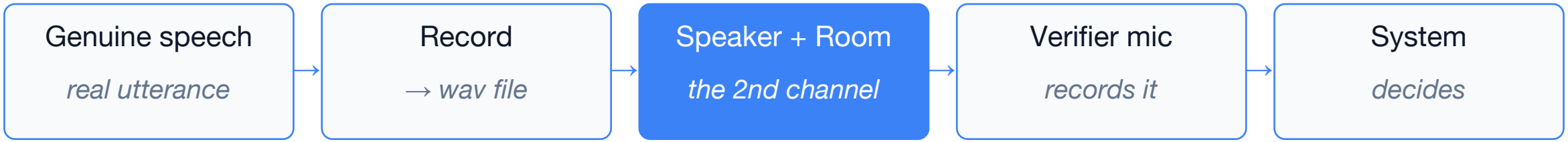
Step 4. Threshold check: only b (0.999) ≥ 0.95.

Answer. Only clip b passes (cos 0.999); a = 0.894 and c = 0.949 are rejected.

Worked challenge — multi-step quantitative. cos(θ)=e·v/(|e||v|) — slides p21–22.

Replay Attack — Record a Real Voice, Play It Back

A replay attack is the simplest spoof: record a genuine human utterance, then play it back to the voice verifier. No AI is used — the voice is 100% real, so words, prosody and timbre are genuine. It fools any system that only asks ‘does this match the enrolled voice?’



Classification — by playback chain (easy → hard to detect)

Phone → phone

- Tiny speaker, compressed, band-limited.
- Strongest channel coloration.
- Easiest to detect.

Speaker, quiet room

- External / Bluetooth speaker.
- Cleaner playback, less coloration.
- Medium difficulty.

Studio monitor + mic

- Hi-fi playback, low reverb.
- Weak channel signature.
- Hardest to detect.

Source: ASVspoof 2017 Physical-Access (PA) replay corpus; Chen et al. (2015) CQCC features

The Second Channel — How a Replay Gives Itself Away

The replayed voice is genuinely human, so there is nothing synthetic to find. The one clue a replay cannot hide: it was played through a loudspeaker and a room — a second acoustic channel that imprints its own signature on the sound.

Loudspeaker

- Band-limits and colors the sound.
- A mic never records a speaker cleanly.
- → timbre shifts, high frequencies cut.

Room

- Adds reverberation.
- Echoes of the replay room.
- → smeared, hollow reverb tail.

Recorder

- The verifier mic captures all of it.
- Double coloration on top of the voice.
- → the channel fingerprint.

Detection & defence: classifiers (CQCC/MFCC + GMM, or neural nets) learn this channel signature — a live genuine utterance lacks it, so the replay is flagged. The strongest fix is liveness / challenge-response, asking the speaker for something a recording cannot give.

Source: Todisco et al. (2017) CQCC-GMM for replay detection arXiv:1710.06635; liveness / challenge-response countermeasures

Replay — Signal Model

Belongs to: Replay attack — why a replay is detectable even though the voice is genuinely human.

$$y(t) = (h * x)(t) + n(t)$$

Symbols

y = mic signal; x = real recording; h = speaker + room; n = noise; * = convolve.

How to read it

Replay passes x through a speaker+room (convolve with h) and adds noise n — that channel is the only fingerprint.

Example: Mount a Replay Attack

What the code does

- Record: take a genuine (bona-fide) utterance.
- Channel: convolve with speaker + room impulse response h.
- Noise: add ambient noise n → the re-recorded clip.
- Detect side: countermeasures look for reverb / coloration, not synthesis.

replay_attack.py

```python
# Mount a replay attack: re-record a real voice through a speaker+room
import soundfile as sf, numpy as np
from scipy.signal import fftconvolve

bona_fide, sr = sf.read("victim_enrollment.wav")

# replay channel = loudspeaker + room impulse response h, plus noise n
#    y(t) = (h * x)(t) + n(t)
h_room = np.load("room_ir.npy")
replayed = fftconvolve(bona_fide, h_room)
replayed = replayed[:len(bona_fide)]
replayed += np.random.normal(0, 1e-3, replayed.shape)

sf.write("replay_attempt.wav", replayed.astype(np.float32), sr)
```

Listen: original vs. algorithm output

Original recording

Replayed (speaker+room)

Question 2

A bank checks who you are by your voice. Two fraud attempts:

(A) The fraudster plays a REAL recording of the customer — through a laptop speaker — into the phone.

(B) The fraudster sends an AI-generated voice file of the CEO through the bank's website.

For each: is it a Replay attack or a Logical-Access attack? And what clue should the detector look for?

Answer 2

(A) Replay — a real recording is just played back. The voice is genuine, so look for the loudspeaker+room effects: reverb, muffling, noise.

(B) Logical Access — a fake file sent straight in. It never touched a mic, so look for synthesis traces: unnatural spectrum / phase.

Replay Attack — High-Frequency Loss in Decibels

Problem. A replay loudspeaker band-limits the signal so the clip keeps only 18% of its original high-frequency energy. (a) Express this loss in decibels. (b) A detector flags any clip whose high-frequency loss exceeds 6 dB — is this replay flagged? (c) The room adds reverberation with T_{60} = 0.4 s; for a 1.2 s clip, how many reverb 'tails' smear it?

Worked solution

Step 1. Loss = $10 \cdot \log_{10}(0.18) = 10 \times (-0.745) = -7.45$ dB

Step 2. Magnitude of loss = 7.45 dB, which is > 6 dB → flagged.

Step 3. Reverb tails = clip duration / T_{60} = 1.2 / 0.4 = 3.

Step 4. So the replay is both band-limited (–7.45 dB) and smeared by ~3 tails.

Answer. –7.45 dB loss → flagged; ~3 reverb tails smear the 1.2 s clip.

Worked challenge — multi-step quantitative. Replay signal model $y = h * x + n$ — slide p25.

MODULE 1

04

1:35 – 2:05 · 30 min

Passive Detection Methods & Datasets

Traditional vs learning-based detection; ASVspoof LA / PA / DF, WaveFake, In-the-Wild, ADD.

Section 4 of 7

What is Passive Detection?

Passive = the detector only listens to the audio and decides real or fake. It cannot challenge the speaker (unlike active methods that ask for a random phrase).

Why passive matters

- No interruption: a bank's voice lock must verify a real call without asking 'please read this random sentence'.
- Real-time: the decision happens in milliseconds, on a single clip of audio.
- Adversarial: the attacker knows the detector exists and tries to evade it — so the detector must generalize beyond known attacks.

Artifacts — Three Dimensions a Fake Cannot Hide

A deepfake is never a perfect copy — it leaves measurable traces (artifacts) in three independent dimensions, taken apart in the pages that follow.

Spectral

The frequency make-up of the voice. A vocoder fake can't rebuild the fine high-frequency texture a real voice has — its highs look smoothed.

- Cue: missing high-freq detail.
- Detect: CQCC / spectrogram CNN.

Temporal

How the sound changes over time. A deepfake over-smooths the sharp transitions and tiny pitch jitter of real speech.

- Cue: smeared transitions.
- Detect: F0 / modulation features.

Phase

The exact timing alignment within each cycle. Generators ignore phase, so reconstruction leaves a buzzy incoherence.

- Cue: phasey / metallic buzz.
- Detect: phase / raw-waveform net.

Next: Spectral → Temporal → Phase — each gets a concept page and a listen page (p27–p32); summary on p33.

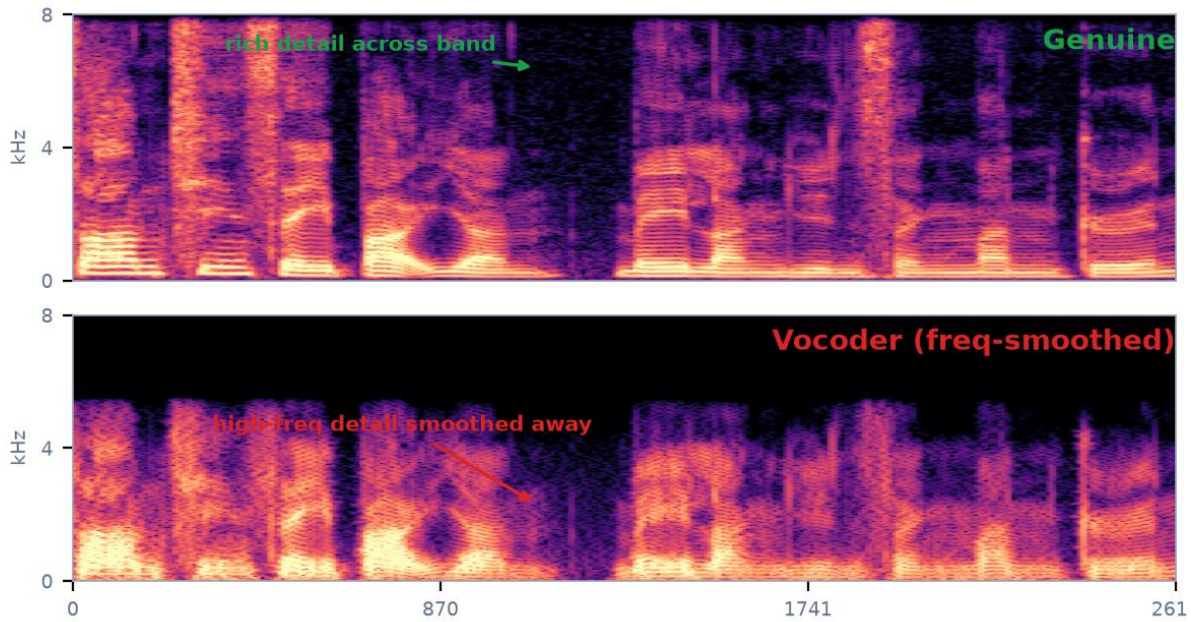
Source: Patel & Patil (2017) deepfake artifacts; Bonetti & Nanni phase cues; ASVspoof countermeasures exploit these three classes

Spectral Artifacts — Where the Fine Detail Vanishes

A spectrogram-based deepfake rebuilds sound from a coarse set of spectral bands. It cannot recreate the fine, high-frequency texture a real vocal tract always produces — so the upper part of its spectrogram looks smoothed and empty.

WHY IT HAPPENS

- Low resolution at high freq — mel-bands & neural vocoders average the highs.
- Vocoder resynthesis blurs fine harmonic / formant structure.
- Listen for: missing 'air', softened s, sh, f sounds.



Magnitude spectrogram of REAL speech. The vocoder fake loses detail above ~3 kHz.

Source: Spectral artifacts of neural vocoders; ASVspoof LA countermeasures (CQCC / spectrogram CNN)

Hear the Spectral Artifact

A real human recording vs. the same speech put through the deepfake step that breaks this dimension.

Genuine · real recording

Deepfake · vocoder resynthesis

WHAT TO LISTEN FOR

- Genuine: bright, crisp, full of air.
- Deepfake: duller, muffled, the highs washed out.
- Same words — the fake lost the sparkle.

Takeaway: this washed-out high end is exactly the spectral signature a detector flags.

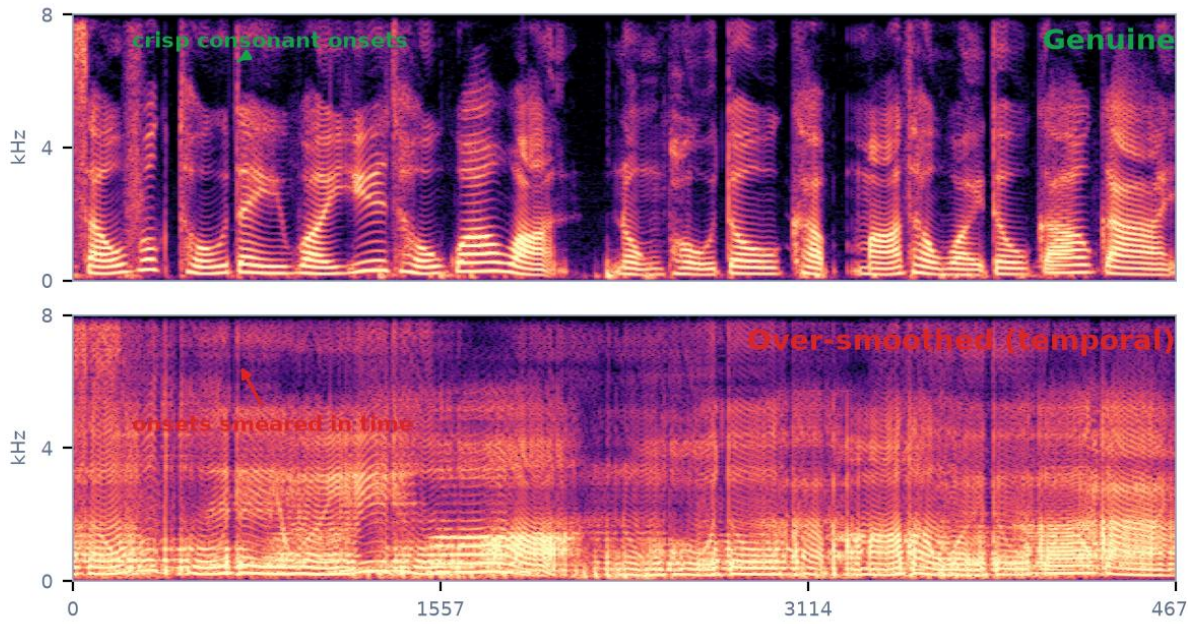
Source: Audio: real human speech (16 kHz corpus) · converted with the actual deepfake DSP (STFT magnitude manipulation / Griffin-Lim) to isolate each artifact

Temporal Artifacts — A Pulse That's Too Smooth

Real speech carries fast changes — the crisp onset of a 't', the snap of a 'k'. A deepfake trained for smoothness over-smooths these in time, so the transitions blur together and the micro-randomness of a real voice flattens out.

WHY IT HAPPENS

- Generators blur fast temporal modulations.
- Consonant onsets & micro-jitter get averaged away.
- Listen for: slurred, lispy, 'watery' transitions.



Spectrogram of REAL speech. The over-smoothed fake smears onsets across time.

Source: Temporal over-smoothing in neural TTS/VC; modulation-frequency features capture it

Hear the Temporal Artifact

A real human recording vs. the same speech put through the deepfake step that breaks this dimension.

Genuine · real recording

Deepfake · temporal over-smoothing

WHAT TO LISTEN FOR

- Genuine: sharp consonants, lively detail.
- Deepfake: slurred, smeared, slightly watery.
- The blurring of fast changes is the cue.

Takeaway: detectors measure this modulation sharpness; transitions that smear read as synthetic.

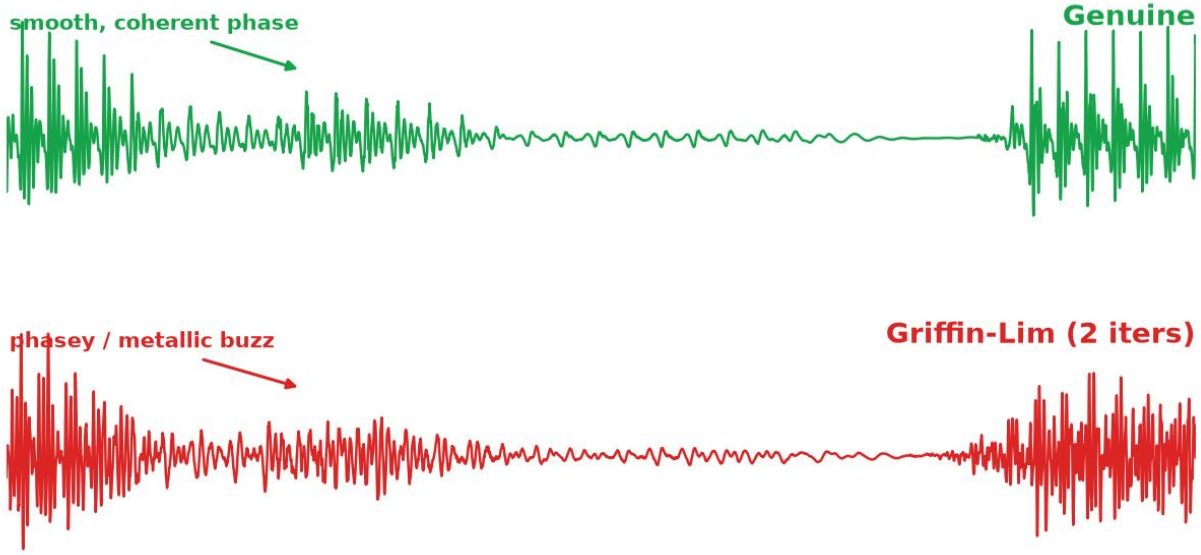
Source: Audio: real human speech (16 kHz corpus) · converted with the actual deepfake DSP (STFT magnitude manipulation / Griffin-Lim) to isolate each artifact

Phase Artifacts — When the Waveform Won't Line Up

Phase is the exact timing offset within each cycle. Humans are nearly deaf to absolute phase, so generators neglect it. But reconstructing speech from a magnitude spectrogram (Griffin–Lim, overlap-add) leaves phase incoherence — a buzzy, metallic edge a person barely hears.

WHY IT HAPPENS

- Griffin–Lim / vocoder hops guess the phase, imperfectly.
- Concatenation joins frames that don't align in phase.
- Listen for: a buzzy, metallic, 'phasey' tinge.



Waveform zoom of REAL speech. The Griffin–Lim fake shows incoherent, phasey cycles.

Source: Phase as a deepfake cue; STFT / Griffin–Lim phase reconstruction artifacts

Hear the Phase Artifact

A real human recording vs. the same speech put through the deepfake step that breaks this dimension.

Genuine · real recording

Deepfake · Griffin–Lim (2 iters)

WHAT TO LISTEN FOR

- Genuine: a clean, coherent waveform.
- Deepfake: buzzy / metallic / phasey.
- Subtle to a person — loud to a model.

Takeaway: people ignore phase, so it is one of the strongest tells a detector exploits.

Source: Audio: real human speech (16 kHz corpus) · converted with the actual deepfake DSP (STFT magnitude manipulation / Griffin-Lim) to isolate each artifact

From Artifacts to Detectors

Each artifact dimension maps to a family of detector features. The next two pages turn those features into a working real/fake classifier.

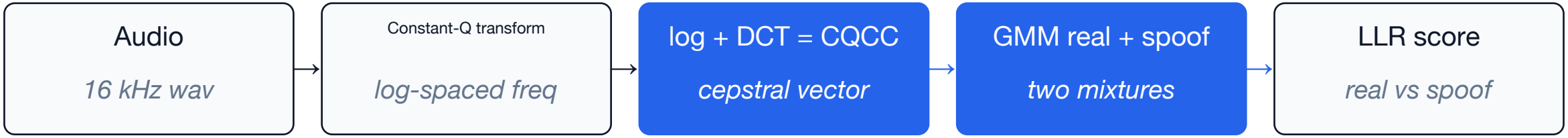
Dimension	The artifact — the cue	Detector we study
Spectral	Smoothed, missing high-frequency detail	CQCC + GMM (p34)
Temporal	Over-smoothed transitions	Wav2Vec2 (Lecture2)
Phase	Incoherent / phasey waveform	AASIST (Lecture2)

In depth: three detectors — CQCC + GMM (p34–35) → AASIST (Lecture2) → Wav2Vec2 (Lecture2).

Source: three detectors studied in depth: CQCC+GMM, AASIST, Wav2Vec2 (p34–39)

CQCC + GMM — The Classic Countermeasure

The ASVspoof baseline: hand-crafted CQCC features + a Gaussian-Mixture classifier per class. Simple, fast, interpretable — the reference every neural model is compared against.



What is CQCC?

- Constant-Q transform — log-spaced frequency bins (octaves), so low frequencies get fine resolution where speech detail lives.
- Then log-power and a DCT → cepstral coefficients.
- Why it works — this fine spectral resolution exposes the smoothing a vocoder leaves; MFCC is too coarse.

Why a Gaussian Mixture?

- GMM — a weighted sum of Gaussians modelling each class's feature distribution.
- Score — log-likelihood ratio LLR = log P(real) − log P(spoof).
- Decide — LLR > threshold τ ⇒ real, else spoof.
- Result — ~9–11% EER on ASVspoof 2019 unseen attacks.

Source: Todisco et al. (2017) CQCC; Reynolds GMM; ASVspoof 2015–19 baseline countermeasure

CQCC + GMM — in Code

Extract CQCC, fit two GMMs, score by log-likelihood ratio.

cqcc_gmm.py

```python
# CQCC + GMM spoof detector  (scikit-learn)
from sklearn.mixture import GaussianMixture
from cqcc import cqcc                        # constant-Q cepstral coefficients
import numpy as np

def feat(x, fs):                             # 1. CQCC features per clip
    c, T = cqcc(x, fs, num_ceps=30)
    return np.mean(c, axis=0)                #    average over time -> 1 vector

X_real  = [feat(x, fs) for x in real_clips]
X_spoof = [feat(x, fs) for x in spoof_clips]

gmm_real  = GaussianMixture(64).fit(X_real)   # 2. one model per class
gmm_spoof = GaussianMixture(64).fit(X_spoof)

def score(x):                                  # 3. log-likelihood ratio
    f = feat(x, fs)[None, :]
    return gmm_real.score(f) - gmm_spoof.score(f)

decision = "real" if score(new_clip) > tau else "spoof"   # 4. threshold
```

Source: cqcc package; sklearn GaussianMixture; LLR scoring — ASVspoof baseline

ASVspoof — The Benchmark Series

Where the detectors above are judged. Each edition added harder attacks, and the eval set always holds unseen spoofs — testing the generalization the artifact lesson (p33) said matters.

Why this benchmark design matters

- LA · PA · DF — Logical Access (TTS/VC), Physical Access (replay), DeepFake: three sub-tasks mapping to the three attack types.
- Unseen-eval — the EVAL set uses spoof systems absent from training, so a detector that merely memorized training attacks fails here.

Link: a detector that watches all three artifact dimensions (p33) is exactly what survives an unseen-eval benchmark.

Source: ASVspoof 2015–2024 challenge series; Wu et al. 2019, Yamagishi et al. 2021

MODULE 1

05

2:05 – 2:35 · 30 min

Evaluation & Accuracy

Section 5 of 7

EER — Equal Error Rate

Belongs to: Detection — evaluation — the metric that scores how good a spoofing countermeasure is.

$$\text{EER}: \quad P_{FA}(\tau^*) = P_{FR}(\tau^*)$$

Symbols τ = threshold; P_FA = false-accept rate; P_FR = false-reject rate.

How to read it Tune τ until false-accepts equal false-rejects — that shared rate is the EER; lower = better.

Computing EER in Code

How to get EER from raw scores:

- Step 1: feed all test-clip scores + true labels to roc_curve.
- Step 2: find the point where FAR (fpr) = FRR (1 – tpr).
- Step 3: that rate is the EER; the corresponding threshold is τ*.
- Tip: AUC = area under the full ROC curve — report both.

compute_eer.py

```python
# Compute EER and AUC from detection scores
from sklearn.metrics import roc_curve, auc
import numpy as np

# labels: 1 = bonafide, 0 = spoof
# scores: higher = more likely bonafide
fpr, tpr, thresholds = roc_curve(labels, scores)

# EER: the point where fpr == 1 - tpr
eer_idx = np.argmin(np.abs(fpr - (1 - tpr)))
eer = fpr[eer_idx]
eer_thresh = thresholds[eer_idx]

# AUC: area under the ROC curve
auc_score = auc(fpr, tpr)

print(f"EER = {eer:.1%}  at threshold {eer_thresh:.3f}")
print(f"AUC = {auc_score:.3f}")
```

Question 3

A fake-voice detector scores every clip 0–1. A cut-off τ decides: score $\geq \tau \Rightarrow$ accepted as REAL.

Two mistakes can happen: False-Accept (a fake slips through) and False-Reject (a real clip rejected).

$\tau = 0.40 \rightarrow$ 20% of fakes accepted, 5% of reals rejected.

$\tau = 0.60 \rightarrow$ 5% of fakes accepted, 20% of reals rejected.

The EER is the point where the two error rates are equal. Find the EER.

Answer 3

As τ rises from 0.40 to 0.60:

fake-accepts fall 20% → 5%, and real-rejects rise 5% → 20% (both move 15 points over $\Delta\tau = 0.20$).

They move at the same speed, so they cross in the middle:

$\tau \approx 0.50$, where both error rates equal $\approx 12.5\%$.

EER = 12.5%

EER from a Score Table

Problem. Eight test clips (4 real R, 4 spoof S), sorted high → low (score ≥ τ ⇒ accepted as REAL): 0.93 R, 0.85 R, 0.72 R, 0.68 S, 0.64 R, 0.58 S, 0.41 S, 0.22 S. Sweep the threshold τ and find the EER (the point where false-alarm rate FAR = false-reject rate FRR).

Worked solution

Step 1. FAR = P(score ≥ τ | spoof); FRR = P(score < τ | real).

Step 2. Try τ ≈ 0.66 (between 0.68 S and 0.64 R): spoofs ≥ τ = {0.68} → FAR = 1/4 = 0.25.

Step 3. Reals < τ = {0.64} → FRR = 1/4 = 0.25.

Step 4. At τ ≈ 0.66 the two error rates are equal → that rate is the EER.

Answer. EER = 25% at τ ≈ 0.66 (one real and one spoof cross over at the boundary).

Worked challenge — multi-step quantitative. EER = the shared rate where FAR = FRR — slide p42.

Why Cross-Dataset Detection Fails

A model trained on ASVspoof 2019 can score 0.83% EER — but test it on 2021 and EER jumps to ~17%. Why?

Unseen TTS

Each TTS system leaves a different spectral fingerprint. A model trained on WaveNet + Tacotron spoofs may miss spoofs from VITS or a brand-new system it has never seen.

Compression

Real-world audio goes through mp3, opus, or AMR codecs. These add quantization noise that MASKS the subtle artifacts the detector learned to spot.

Domain shift

Different recording devices, languages, accents, noise conditions. The statistical distribution of the test data differs from training — the model's assumptions break.

MODULE 1

06

2:35 – 2:50 · 15 min

Guided Lab 1

Extract features, train a baseline detector, visualize real vs fake.

Section 6 of 7

Lab — Build a Deepfake Detector

A self-contained Colab notebook — it synthesizes its own bonafide vs spoof clips, so it runs with no downloads. Your job: extract features, train a classifier, score, and report the EER.

Your four tasks

- Task 1 — Features: take 13 MFCC coefficients per clip (torchaudio), averaged over time.
- Task 2 — Train: fit one Gaussian (mean + covariance) per class on 200 train clips.
- Task 3 — Score: score 200 test clips by log-likelihood ratio LLR = log P(spoof) − log P(real).
- Task 4 — Evaluate: sweep a threshold; find the EER where false-alarm rate = false-reject rate.

Setup

- Open Colab → paste the code (p41).
- Needs only numpy, scipy, torchaudio — all preinstalled.

Deliverable

- One number: the EER.
- A DET curve (false-alarm vs false-reject).
- Compare with the reference answer on p42.

Source: Self-contained synthetic dataset (no download); mirrors the spectral-artifact idea from p27

Lab — Reference Code (Colab-ready)

Copy into one Colab cell → Run. Output: EER = 12.0%

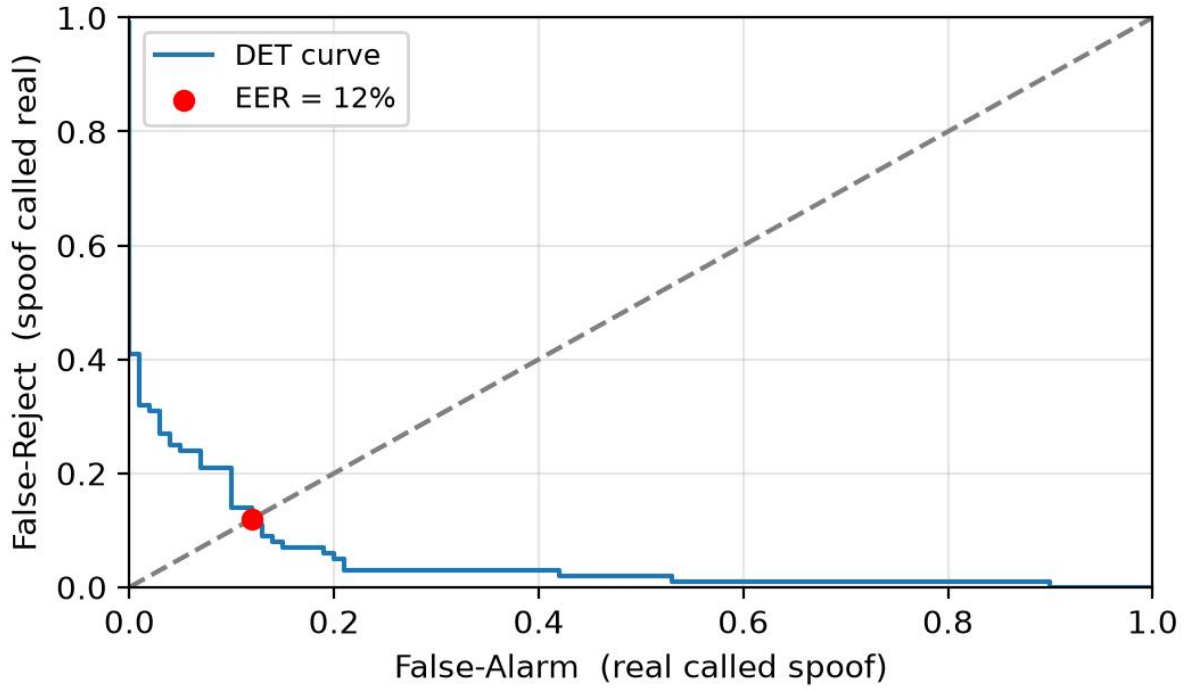
lab_deepfake_detector.py

```
# Lab: Audio Deepfake Detector — MFCC -> Gaussian -> EER  (Colab, no downloads)
import numpy as np, torch, torchaudio
from scipy.signal import butter, filtfilt
SR=16000
MFCC=torchaudio.transforms.MFCC(SR,n_mfcc=13,melkwargs=dict(n_fft=512,hop_length=160,n_mels=40))
np.random.seed(0)

def synth(spoof,n=16000):                       # 1. bonafide vs spoof vowel clip
    t=np.arange(n)/SR; ph=2*np.pi*np.cumsum(np.random.uniform(95,135)+4*np.sin(2*np.pi*4*t))/SR
    s=sum(0.7*np.exp(-((h*120-700)/150)**2)*np.sin(h*ph) for h in range(1,40))+np.random.randn(n)*0.08
    r=np.random.rand()
    if spoof:    cut=np.random.uniform(6000,7400) if r<0.30 else np.random.uniform(3000,4600)
    elif r<0.18: cut=np.random.uniform(5200,6800)        # a few reals are slightly muffled
    else: cut=None
    if cut is not None: s=filtfilt(*butter(4,cut/(SR/2)),s)
    return s.astype('f')

def feat(x): return MFCC(torch.from_numpy(x)).mean(1).numpy()      # 2. MFCC feature
def mk(N): y=np.array([i%2 for i in range(N)]); return np.array([feat(synth(b)) for b in y]), y
Xtr,ytr=mk(200); Xte,yte=mk(200)

def fit(X): return X.mean(0), np.cov(X,rowvar=False)+np.eye(13)*1e-3
def lp(X,m,c):
    _,ld=np.linalg.slogdet(c); d=X-m
    return -0.5*(13*np.log(2*np.pi)+ld+np.einsum('ij,jk,ik->i',d,np.linalg.inv(c),d))
mr,cr=fit(Xtr[ytr==0]); ms,cs=fit(Xtr[ytr==1])
score=lp(Xte,ms,cs)-lp(Xte,mr,cr)               # 3. LLR (high => spoof)
sr_,ss_=score[yte==0],score[yte==1]; best=eer=1.0
for t in np.unique(score):                      # 4. sweep threshold -> EER
    far,frr=np.mean(sr_>=t),np.mean(ss_<t)
    if abs(far-frr)<best: best=abs(far-frr); eer=far
print(f"EER = {eer*100:.1f}%")
```

Source: Verified runnable: numpy + torchaudio + scipy; prints EER = 12.0% on a CPU laptop / Colab

Lab — Reference Answer

Reference result: EER ≈ 12% — a realistic, non-trivial number (not 0%).



Why ≈ 12% and not 0%?

- Mild spoofs (cutoff 6–7.4 kHz) look nearly real → false rejects.
- A few muffled reals → false alarms.
- The overlap IS the EER.

How to push EER lower

- Richer features: CQCC / raw spectrogram.
- Stronger model: GMM-64 or a small neural net.
- Real data: swap in ASVspoof clips.

Source: DET curve from the exact code on p41; EER = false-alarm where FAR = FRR

ASVspoof 2019 — LA & PA Tasks, Deep Dive

LA — Logical Access (synthetic speech)

- Train / Dev: 6 known spoof systems (WaveNet, Tacotron, FastSpeech, neural VC…). The model learns these.
- Eval: 13+ UNSEEN systems — completely new TTS/VC the model has never seen.
- Design: eval tests generalization, NOT memorization.
- Scale: ~25k clips (train), ~25k (dev), ~71k (eval).
- This is the most cited anti-spoofing benchmark.

PA — Physical Access (replay)

- Setup: genuine speech is played through a loudspeaker into a mic, in a real room.
- Variety: 26+ loudspeaker + room + microphone combinations.
- Train / Dev: known replay configurations.
- Eval: unseen replay setups (different speakers, rooms, distances).
- Tests: can the detector catch re-recorded audio, not just synthesis?

ASVspoof 2021 — Harder: Compression & Deepfakes

Three tasks, each tougher than 2019:

LA	Re-test of 2019 LA — same spoof systems, BUT audio now passes through codecs (mp3, opus) and transmission effects.
PA	Replay attacks — harder rooms, more speaker/mic combinations, background noise.
DF (new)	Deepfake speech — entirely new synthetic systems not in 2019. A fresh generalization test.

THE KEY FINDING

Train on 2019 LA (EER ~1%), test on 2021 LA → EER jumps to ~17%. Compression + unseen attacks = much harder.

Results — Who Wins on ASVspoof 2019 LA?

How detection quality improved over the years (EER on ASVspoof 2019 LA):

GMM + CQCC	~9–11%	Hand-crafted features + statistical model. The official baseline.
LCNN + CQCC	~5%	Light CNN learns from CQCC spectrograms.
RawNet2	~2–3%	End-to-end on raw waveform — no feature engineering.
AASIST	~0.83%	Spectro-temporal graph attention. State of the art (2022).
Wav2Vec2 + classifier	~0.5–1%	Self-supervised pre-training on millions of hours of speech.

Lesson: learned features (neural / self-supervised) beat hand-crafted ones — and the gap grows on UNSEEN attacks.

Feature Landscape — What the Classifier Sees

Four ways to turn audio into model input — each trades interpretability for performance.

CQCC	Spectrogram	Raw waveform	SSL embeddings
Hand-crafted cepstral coefficients	Visual time-frequency image	Direct samples (no extraction)	Pre-trained (Wav2Vec2)
Traditional GMM baseline. Interpretable but fixed.	Fed to a CNN — the model learns spatial patterns. Most common neural input.	SincNet / RawNet learn their own filters. No human feature design.	Rich representations from millions of hours of speech. Fine-tune for detection.

Trend: left → right = more automatic, more data-hungry, better on unseen attacks.

Logical Access — Synthetic Speech, Injected Digitally

What it is

- Delivery: a TTS/VC-generated .wav file fed straight to the verifier API — no microphone, no room.
- Attack systems (ASVspoof 2019 LA): WaveNet, Tacotron, FastSpeech, neural VC, etc. — 6 known + 13 unseen in eval.
- The signal is 100% synthetic— there is no genuine human component.
- No channel artifacts→ the detector must catch the SYNTHESIS itself.

Four artifacts LA spoofs leave behind

- Spectral: over-smooth formants; vocoder cuts the high band → dull / metallic.
- Temporal: irregular phoneme durations; autoregressive alignment slips → stutter.
- Phase: magnitude-only vocoders invent phase (Griffin-Lim) → inconsistencies.
- Prosodic: pitch (F0) too flat; stress / rhythm too regular; missing emotion.

All four are exploitable because LA has no channel to hide behind.

Physical Access — Replay Through a Speaker + Room

What it is

- Delivery: genuine speech played through a loudspeaker into the verification mic.
- The signal IS real human speech— words, prosody, timbre are genuine.
- ASVspoof 2019 PA: 26+ loudspeaker × room × microphone combinations.
- No synthesis artifacts→ the detector must catch the CHANNEL, not the synthesis.

Artifacts PA spoofs leave behind

- Reverb: room impulse response adds echo tails and smearing.
- Coloration: the loudspeaker's frequency response tints the signal.
- Noise: ambient room noise + microphone self-noise.
- Double encoding: the original was already digitized; re-recording adds a 2nd layer.

Signal model: $y(t) = (h * x)(t) + n(t)$ — detect h and n, not x.

DeepFake — The Newest, Hardest Category

What it is

- New in ASVspoof 2021: deepfake speech from entirely new, more advanced TTS/VC systems.
- Difference from LA: DF systems are newer, more natural, harder to distinguish from genuine.
- Real-world: TikTok voice clones, political deepfakes, scam calls.
- Compression added: DF clips go through codecs (mp3, opus) → masks subtle artifacts.

Why DF is the hardest

- Better synthesis: the latest TTS leaves fewer spectral / phase traces.
- Compression: codec effects mask whatever traces remain.
- Cross-domain: models trained on 2019 LA fail on 2021 DF → EER jumps from ~1% to ~20%+.
- Open problem: generalizable DF detection is an active research frontier.

Dataset Summary — Where to Get the Data

All links verified July 2024. Download from the official sites (some require registration).

Dataset	Year	Tasks	Scale	Where
ASVspoof 2019	2019	LA + PA	~20k train / ~71k eval	asvspoof.org (Zenodo / Datashare)
ASVspoof 2021	2021	LA + PA + DF	similar + compression	asvspoof.org (Zenodo: 4835108)
WaveFake	2021	6 TTS systems	104k+ clips	github.com/RUB-SysSec/wavefake
ADD 2023	2023	FG + RL + AR	in-the-wild	addchallenge.cn (arXiv:2305.13774)
AT-ADD	2024	robustness + generalization	real-world	at-add.com (Codabench)

MODULE 1

07

2:50 – 3:00 · 10 min

Debrief, Key Takeaways & Preview

Lab observations; Module 2 preview (raw waveform & AASIST); homework.

Section 7 of 7

Debrief & Module 2 Preview

Lab observations

- Hand-crafted features work —MFCC + MLP catches obvious fakes (~10% EER).
- But they generalize poorlyto unseen attack types.
- Neural models are betterbut need more data and compute.
- Next step:go beyond spectrograms — use the raw waveform.

Module 2 preview + homework

- Module 2:raw waveform → SincNet → RawNet → AASIST.
- Goal:end-to-end learning, no hand-crafted features.
- Homework:read the AASIST paper (Jung et al., 2022).
- Brainstorm:a project with a societal angle (e.g., NGO fact-checking tool).

Full Detection Pipeline — Copy & Run

colab_deepfake_demo.py

```
# === Audio Deepfake Detection — Full Colab Pipeline ===
# 1. Setup
!pip install librosa soundfile scikit-learn

import librosa, numpy as np
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import roc_curve, auc

# 2. Extract MFCC features from each clip
def features(path):
    y, sr = librosa.load(path, sr=16000)
    return librosa.feature.mfcc(y=y, sr=sr, n_mfcc=20).mean(axis=1)

X = np.array([features(f) for f in clip_paths])
y = np.array(labels)                    # 1 = bonafide, 0 = spoof

# 3. Split & train
from sklearn.model_selection import train_test_split
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3)
clf = MLPClassifier(hidden_layer_sizes=(64, 32), max_iter=300)
clf.fit(X_tr, y_tr)

# 4. Evaluate: EER + AUC
scores = clf.predict_proba(X_te)[:, 1]
fpr, tpr, thresh = roc_curve(y_te, scores)
eer = fpr[np.argmin(np.abs(fpr - (1 - tpr)))]
print(f"EER = {eer:.1%}   AUC = {auc(fpr, tpr):.3f}")
```

Worked Example — Computing EER from Scores

You have 6 test clips. Scores (higher = more likely real) and true labels:

Clip A: 0.92 (real) Clip B: 0.71 (real) Clip C: 0.45 (fake)

Clip D: 0.83 (real) Clip E: 0.18 (fake) Clip F: 0.61 (fake)

Sort low to high: E=0.18(fake), C=0.45(fake), F=0.61(fake), B=0.71(real), D=0.83(real), A=0.92(real)

At threshold 0.66: accepts {B,D,A} = 3 real, rejects F(0.61,fake). FAR=0%, FRR=0%.

At threshold 0.50: accepts {F,B,D,A} but F is fake! FAR=33%, FRR=0%.

EER is where FAR = FRR. Here scores barely overlap, so EER is near 0%.

In real systems scores overlap heavily; EER of 1-10% is typical.

Worked Example — ASVspoof Generalization Gap

Scenario: Your team trains a CNN on ASVspoof 2019 LA train (6 known spoof systems).

Results: EER on train = 1.2% | EER on eval = 14.8%

Q1: Why is eval EER so much higher?

A1: Eval has 13+ UNSEEN systems. Model memorized the 6 known fingerprints but cannot generalize.

Q2: Boss says add eval systems to training. Why wrong?

A2: Eval tests generalization. Train on eval = testing memorization, not robustness.

Q3: How to improve 14.8%?

A3: Augmentation (codecs, noise), Wav2Vec2 features, or multi-dataset training.

Worked Example — Classify the Attack

Three voice clips arrive at a bank voice system:

(A) Customer real voicemail played through laptop speaker into phone.

(B) AI voice file of CEO sent via bank website API.

(C) Live call with real-time voice changer to sound like customer.

Answers:

(A) Replay — catch the channel (reverb, coloration).

(B) Logical Access — catch synthesis artifacts (spectral, phase).

(C) Logical Access (VC) — catch synthesis + latency artifacts.

Key Takeaways

Synthetic voice is cheap and convincing.

TTS turns text into speech; VC swaps the speaker from seconds of audio — good enough to fool the ear and the phone.

A fake reaches the system in one of two ways — and that decides how you catch it.

A digital file (Logical Access) has no mic/room to hide behind → catch the synthesis. A re-played real recording (Replay) sounds human → catch the channel.

Detection is a yes/no classifier scored by EER.

Extract features → score → compare to a threshold; EER is where wrong-accepts equal wrong-rejects — lower is better.

References

1. van den Oord, A., et al. (2016). WaveNet: a generative model for raw audio. *Preprint.*

2. Shen, J., et al. (2018). Natural TTS synthesis by conditioning WaveNet on mel-spectrogram predictions. *Proc. ICASSP.*

3. Ren, Y., et al. (2019). FastSpeech: fast, robust and controllable text to speech. *Proc. NeurIPS.*

4. Kim, J., et al. (2021). A conditional variational autoencoder with adversarial learning for end-to-end TTS (VITS). *Proc. NeurIPS.*

5. Wang, C., et al. (2023). VALL-E: neural codec language models for zero-shot text-to-speech synthesis. *Preprint.*

6. Le, M., et al. (2023). Voicebox: text-guided multilingual universal speech generation. *Preprint.*

7. Ao, J., et al. (2022). SpeechT5: unified-modal encoder-decoder pre-training for spoken language processing. *Proc. ACL.*

8. Hunt, A. J., & Black, A. W. (1996). Unit selection in a concatenative speech synthesis system using a large acoustic database. *Proc. ICASSP.*

9. Griffin, D. W., & Lim, J. S. (1983). Signal estimation from modified short-time Fourier transform. *IEEE Trans. ASSP, 31(2), 236–243.*

Continued on next page →

References

10. Kameoka, H., et al. (2018). StarGAN-VC: non-parallel many-to-many voice conversion using StarGAN. *Proc. IEEE SLT.*

11. Kaneko, T., & Kameoka, H. (2018). CycleGAN-VC: non-parallel voice conversion using cycle-consistent adversarial networks. *Proc. APSIPA ASC.*

12. Qian, K., et al. (2019). AutoVC: zero-shot voice style transfer with only autoencoder loss. *Proc. ICML.*

13. Snyder, D., et al. (2018). X-vectors: robust DNN embeddings for speaker recognition. *Proc. ICASSP.*

14. Davis, S. B., & Mermelstein, P. (1980). Comparison of parametric representations for monosyllabic word recognition. *IEEE Trans. ASSP, 28(4), 357–366.*

15. Todisco, M., et al. (2017). Constant-Q cepstral coefficients: a spoofing countermeasure for automatic speaker verification. *Computer Speech & Language, 45, 145–156.*

16. Todisco, M., et al. (2019). ASVspoof 2019: future horizons in spoofed and fake audio detection. *Proc. INTERSPEECH.*

17. Yamagishi, J., et al. (2021). ASVspoof 2021: accelerating progress in spoofed and deepfake speech detection. *Proc. ASVspoof Workshop, INTERSPEECH.*

18. Kinnunen, T., et al. (2018). T-DCF: a detection cost function for the tandem assessment of spoofing countermeasures and ASV. *Proc. Odyssey.*